QUICK REFERENCE 3-PAGER • A4

Markdown Summary Tool

Engine computes → LLM narrates → Verifier enforces — grounded markdown from any dataset

Tool **summarize_markdown** · **POST /v1/summarize** · DuckDB push-down · Single cheap LLM call · Mechanical verifier

WHAT IT IS One call: datasets + question → grounded markdown + facts + warnings

■ Deterministic
DuckDB computes **every number** over full data. No sampling. Facts are SQL results, not LLM guesses.

■ Grounded narrative
Single cheap LLM call verbalizes pre-computed facts. Verifier ensures **no hallucinated numbers**.

■ Sanitized
HTML/JS stripped, URIs sanitized, columns redacted. Output is **safe markdown** ready to render.

INPUTS & OUTPUTS

Pydantic v2 strict schemas • extra="forbid"

■ INPUT SummaryRequest

Field	Type / Values	Notes
datasets	list[**DataRef**] (min 1, unique names)	Inline / URL / connector
DataRef.kind	**inline_json • inline_csv • url • connector**	Kind dictates required field
DataRef.name	str 1–128 chars, unique	Used as DuckDB table name
DataRef.data	Any (inline kinds)	JSON array or CSV string
DataRef.url	str (url kind)	Fetched via httpx + pagination
question	str (min 1 char, required)	NL question for the summary
options	**SummaryOptions** (optional)	See options table below

■ OUTPUT SummaryResponse (JSON)

Field	Type	Description
markdown	str	Sanitized markdown narrative (HTML/JS stripped)
facts	list[Fact]	Computed aggregates: count/sum/avg/min/max/group_count/topk/distinct
warnings	list[str]	Non-fatal: truncation, empty, type coercion
artifacts	dict	Optional extras (e.g. chart specs)
meta	dict	elapsed_ms, verified, row_count, fact_count

SummaryOptions All optional • strict, extra="forbid"

Option	Type / Default	Constraints	Effect
max_rows	int | None	ge=1	Cap rows per dataset; warns on truncation
redact	list[str] default []	—	Column names to redact → [redacted]
locale	str default "en-US"	BCP-47	Number/date formatting
timeout_ms	int default 30000	ge=1	Per-stage timeout (ms)
include_warnings	bool default true	—	Include warnings in response

DataRef Kinds How datasets are supplied

inline_json	data: **list[dict]**	JSON array of rows	✔ inline
inline_csv	data: **str** (CSV)	CSV string + options.delimiter	✔ inline
url	url: **str**	Fetched via httpx, pagination	remote
connector	connector_config: **dict**	Connector params (extensible)	extensible

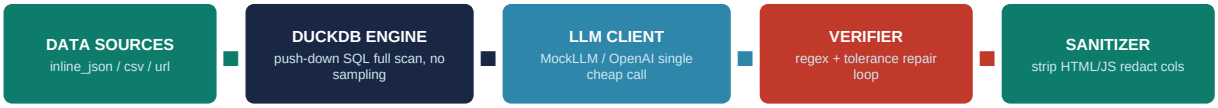
*All DataRef kinds share **name** (unique) + **options** (dict). Validation: inline kinds require **data**, url requires **url**, connector requires **connector_config**.*

ARCHITECTURE & PIPELINE

From raw datasets to sanitized markdown

Architecture Diagram

Engine computes → LLM narrates → Verifier enforces



• **DuckDB push-down** — Every number computed via SQL over **full datasets**. No sampling for final values. COUNT/SUM/AVG/MIN/MAX/GROUP BY.

• **MockLLM / OpenAI** — Single cheap call: **build_prompt(facts)** → LLM verbalizes. Model via **API_BASE/API_KEY** (claude-fable-5.1).

• **Mechanical verifier** — Regex extracts numbers from markdown, checks against facts with **tolerance 1e-9**. Hallucinated numbers flagged.

• **Sanitizer** — Strips HTML tags, JS URIs, event handlers. Redacts columns via **[redacted]**. nh3 or regex fallback.

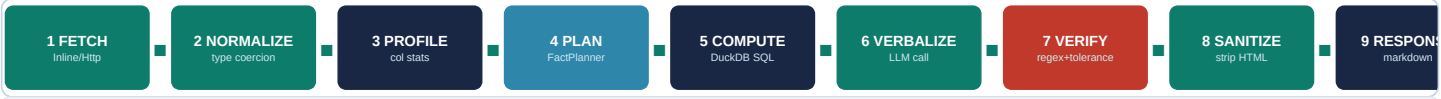
• **FactPlanner** — Heuristics pick facts: row count always, numeric cols → sum/avg/min/max, low-card strings → group_count.

• **Gateway** — FastAPI: **POST /v1/summarize** + **GET /health**. Pydantic strict validation, extra="forbid".

Fig. 1 — Architecture: DuckDB computes facts → LLM narrates → Verifier enforces correctness → Sanitizer ensures safety.

Pipeline Flowchart

9 stages from fetch to response



• **Fetch** — InlineConnector / HttpConnector per DataRef.kind; collect warnings for unknown kinds. *In: DataRef[] → Out: raw rows*

• **Normalize** — Type coercion, null handling, date parsing; truncate to **max_rows** with warning. *In: raw rows → Out: typed rows*

• **Profile** — Column stats, inferred types (str/int/float), cardinality, 5-row sample. *In: typed rows → Out: profile*

• **Plan** — FactPlanner: row count always, numeric → sum/avg/min/max, low-card str → group_count. *In: profile+question → Out: FactSpec[]*

• **Compute** — DuckDBEngine runs SQL per FactSpec over full data. *In: FactSpec[] → Out: Fact[]*

• **Verbalize** — build_prompt(facts) → MockLLMClient.generate() single call. *In: Fact[] → Out: raw markdown*

• **Verify+Repair** — Regex extracts numbers, tolerance check (1e-9), repair loop fixes hallucinations. *In: markdown+facts → Out: verified md*

• **Sanitize** — Strip HTML/JS URIs/event handlers, redact columns. *In: verified md → Out: safe markdown*

• **Response** — Assemble {markdown, facts, warnings, artifacts, meta{elapsed_ms, verified}}. *Out: SummaryResponse*

• **DuckDB push-down** — All aggregates run inside DuckDB; no Python loops over full data. Warnings accumulate each stage.

Fig. 2 — Pipeline: 9 stages. DuckDB handles all aggregation; LLM verbalizes once; verifier + sanitizer guarantee correctness & safety.

TECHNICAL DEEP-DIVE

Planner • Verifier • Security • API • Worked example

FactPlanner Heuristics

How facts are chosen from profile + question

- **Always** — Row count (**count**) for every dataset.
- **Numeric cols** — sum / avg / min / max per numeric column.
- **Low-card strings** — distinct ≤ 50 → **group_count** (top categories).
- **Question bias** — Keywords boost relevant facts: "total"→sum, "average"→avg, "top"→topk, "max"→max.
- **topk** — Top-K rows by value (limit 5) for ranking questions.
- **distinct** — Unique value count for "how many distinct/unique" questions.
- **Dedup** — Facts deduplicated by id; each fact has kind, value, formatted, tolerance.
- **SQL gen** — Each FactSpec → DuckDB SQL: SELECT COUNT(*), SUM(col), AVG(col), etc.

Fact Kinds

8 kinds computed by DuckDB • each fact: {id, kind, value, formatted, tolerance}

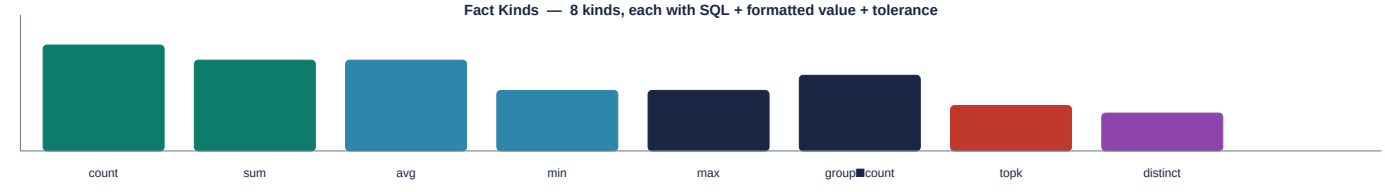


Fig. 3 — Fact kinds: count (rows), sum/avg/min/max (numeric), group_count (categorical), topk (ranking), distinct (unique). All via DuckDB SQL.

Verification Loop

Mechanical check — no LLM re-judging

- **1. Extract** — Regex **_NUMBER_RE** finds all numbers in markdown (handles commas, currency $, %, decimals).
- **2. Check** — Each number compared to facts: formatted variants + **tolerance 1e-9** (relative). Hallucinated numbers flagged.
- **3. Repair** — If mismatch, **repair()** replaces bad numbers with correct formatted values. Re-verify until clean.
- **4. Desc exclusion** — Numbers in fact descriptions excluded from hallucination check (they are expected).

Security

Sanitizer + redaction

- **HTML stripping** — **_HTML_TAG_RE** removes all tags. nh3.clean() if available, regex fallback.
- **JS URIs** — **_JS_URI_RE** strips javascript:/data: URIs.
- **Event handlers** — **_EVENT_HANDLER_RE** removes on*= attributes.
- **Redaction** — Columns in **redact** list → values replaced with **[redacted]** or rows dropped.
- **Strict schemas** — Pydantic extra="forbid", unique DataRef names, min/max constraints.
- **Timeout** — Per-stage **timeout_ms** (default 30s) prevents hanging.

API

Method	Path	Body / Response	Notes
POST	**/v1/summarize**	SummaryRequest → SummaryResponse	Main endpoint; strict validation
GET	**/health**	{"status": "ok"}	Liveness probe
POST	**/v1/summarize** (error)	ErrorResponse {error, detail}	On validation / timeout failure

Worked Mini-Example

Revenue by region → grounded markdown

```
REQUEST  (POST /v1/summarize)
{
  "datasets": [{
    "kind": "inline_json",
    "name": "sales",
    "data": [
      {"region": "EMEA", "revenue": 42000},
      {"region": "APAC", "revenue": 38000},
      {"region": "AMER", "revenue": 51000},
      {"region": "EMEA", "revenue": 15000}
    ]
  }],
  "question": "Summarize revenue by region",
  "options": {"locale": "en-US"}
}
```

```
RESPONSE
{
  "markdown": "## Revenue Summary\n\nTotal revenue is **\$146,000**
\nacross **4** records. EMEA leads with **\$57,000** \n(39%), followed by
AMER (\$51,000) and APAC \n(\$38,000). Average revenue per record:
\n**\$36,500**.",
  "facts": [
    {"kind": "count", "value": 4, "formatted": "4"},
    {"kind": "sum", "value": 146000, "formatted": "146,000"},
    {"kind": "group_count", "value": {"EMEA": 57000, ...}},
    {"kind": "avg", "value": 36500, "formatted": "36,500"}
  ],
  "warnings": [],
  "meta": {"elapsed_ms": 42, "verified": true,
           "row_count": 4, "fact_count": 4}
}
```

Revenue Summary

Total: $146,000 • 4 records • Avg: $36,500

EMEA $57k (39%) ■■■■■■■■

AMER $51k (35%) ■■■■■■■■

APAC $38k (26%) ■■■■■

Sanitized markdown — safe to render

Fact	Value
count	4
sum	146,000
avg	36,500
group_count	EMEA 57k

Fig. 4 — Worked example: 4 rows → 4 facts (DuckDB) → grounded markdown. All numbers verified; HTML stripped.